

How Much Screening Is Enough? An Economic Analysis of Cheap Statistical Screens, LLM Judges, and Strategic Evasion in Detecting Machine-Written Scholarly Text

Abstract

Publishers must decide how much to spend screening documents for machine authorship, yet detection research ignores costs, base rates, and adaptive authors. We develop a decision model of a two-stage screen, in which a free screen decides which documents reach an LLM judge, and extend it to authors who choose how hard to evade. It implies that the screen mainly raises the base rate the judge sees, that a judge with a blind spot is not worth calling, and that a publicly known screen can lose most of its power. We calibrate it with 474 human papers and papers from GPT-4o, Claude, and Llama. A count-based screen detects machine papers well only within a generator, and a strong judge misses Claude. A cascade keeps most detection at lower cost, but a feature-aware edit cuts the screen’s detection from 53% to 8%, weakens the cascade, and does not fool the judge.

Keywords: screening, information asymmetry, machine-generated text, LLM-as-a-judge, strategic evasion, information quality

1. Introduction

Fluent, well-organized prose has long served as a signal of effort in scholarly publishing. Large language models make that signal cheap. A model can draft an abstract, a methods section, or a plausible set of results in a minute, and the output is often good enough to survive a quick read. For publishers, preprint servers, and research-integrity offices, the question is therefore no longer

whether machine-written papers exist. It is how much to spend on finding them, given that every check costs money, every flag costs an investigation, and every miss carries a loss.

The detection literature mostly answers a different question, namely how accurately a classifier separates machine from human text. Accuracy figures are reported for one generator at a time, without a base rate, without the cost of a false alarm or a miss, and without asking whether authors will adapt once they know how they are being screened. Those omissions matter for any organization that has to decide what to deploy. A detector with an excellent hit rate can still be a poor investment if machine papers are rare and its false alarms swamp its detections, and a detector that works today may stop working as soon as its method is published.

We study the screening decision directly. We formulate the organization’s problem as a two-stage screen: a free statistical screen scores every document, and only the highest-scoring documents are sent to a costly second stage, an LLM judge (Zheng et al. 2023). We then extend the model to authors who choose how hard to evade the screen. The model yields propositions about when the judge is worth calling, how deep the screen should reach as the base rate changes, why a judge with a blind spot is worth little, and how a publicly known screen loses power. To make the model concrete, we calibrate it with data. The free screen is built from counts of small linguistic events in English scientific papers, such as parentheses, semicolons, and passive clauses, which we model as a count process. The calibration data are 474 human-written papers from 2015–2019 and matched papers from three generators: GPT-4o, Claude, and Llama.

Three questions organize the paper.

- **RQ1 (Design).** How should an organization combine a free screen with a costly judge, as a function of the base rate, the costs, and the screen’s and the judge’s detection rates?
- **RQ2 (Performance).** How well do a cheap count-based screen and LLM judges detect machine-written papers, and how well do these methods transfer across generators?
- **RQ3 (Evasion).** How much of the screen’s value survives when authors adapt, and which design resists?

We make four contributions. First, we give a decision model of cascaded screening, with closed-form rules for when to call the judge, and we show that the screen’s value comes mainly from improving the precision of the judge’s flags rather than from saving judge fees. Second, we extend the model to strategic authors and show that screens can suffer from a detectability paradox: a screen that is very effective against non-adaptive authors can induce enough evasion to detect less than a weaker screen would. Third, we calibrate both models with data from three generators and two judges, and report results that are uncomfortable for simple detection claims: accuracy does not transfer across generators, a strong judge misses one generator almost completely, and a feature-aware edit removes most of the screen’s power. Fourth, we propose a scale-dependent clustering model of event counts, a doubly stochastic Poisson process, and test it on an open-weight generator that played no part in its development. We do not study whether explainable evidence improves the decisions of human reviewers, which requires a controlled experiment.

2. Background

Screening under asymmetric information. When one party knows more about quality than the other, the less informed party must rely on signals and screens. Akerlof (1970) showed that

quality uncertainty can unravel a market, and Spence (1973) showed that a signal separates types only if it is costly enough that low-quality types cannot imitate it. Reviewing the literature, Connelly et al. (2011) stress the same logic: a signal is informative to the extent that it is costly to fake. For scholarly publishing, fluent writing used to be a costly signal of effort, and language models have lowered that cost. Organizations are left with screens built on traces that authors do not choose deliberately, and the economic question is how much to invest in them and how long they stay informative once authors respond.

Detecting machine-generated text. Existing detectors include supervised classifiers, methods that use a language model’s own probabilities (for example, Gehrmann et al. 2019; Mitchell et al. 2023), and judge-style evaluation by a second model (Zheng et al. 2023). Embedding-based comparisons such as MAUVE (Pillutla et al. 2021) and BERTScore (Zhang et al. 2020) measure whether sets of texts mean similar things and say little about fine-grained behavior. Most detectors are opaque to the person who must act on them, and many deteriorate when the generator changes. We use a deliberately transparent screen so that every quantity traces back to a countable feature.

Information quality and design. Research on information quality treats accuracy and credibility as properties an organization must monitor (Wang and Strong 1996), and the design-science tradition judges an artifact by whether it solves an organizational problem at acceptable cost (Hevner et al. 2004). We follow that orientation: cost per screened document, base rates, and adaptive behavior are first-class elements of the analysis.

Count data and dispersion. For counts of independent events in fixed windows, the Poisson model implies a dispersion index, the variance divided by the mean, of one. Real text rarely satisfies this, because topics, sections, and formatting conventions make events cluster. Count-

data methods accommodate extra variation, for instance through the negative binomial family (Cameron and Trivedi 2013). We read the dispersion index descriptively, as a summary of how clustered a document's events are.

3. A Model of Cascaded Screening and Evasion

3.1 Setting

An organization receives a stream of documents, a share π of which are machine-written. A free screen assigns each document a score s , with likelihood ratio $\ell(s) = f_M(s) / f_H(s)$, where f_M and f_H are the score densities of machine and human papers; we assume ℓ is nondecreasing in s . Given a base rate π , the posterior probability that a document with score s is machine-written is $p(s; \pi) = \pi \ell(s) / (\pi \ell(s) + 1 - \pi)$.

The organization may send a document to a judge. A judged document is flagged with probability J if it is machine-written (the judge's detection rate) and a if it is human (its false-alarm rate), independently of s given the class. Flagged documents are investigated. Four costs apply: c_J , the price of one judge call; c_R , the cost of investigating one flagged paper, assumed to settle its status; L , the loss from each machine paper that is not flagged; and nothing for the screen itself. Documents that are not judged are not flagged. A cascade is therefore a depth: the screen score above which documents are judged.

3.2 When to call the judge

Proposition 1 (when to call the judge). Sending a document with posterior p to the judge reduces expected cost, relative to not sending it, by $V(p) = p \cdot [(L - c_R)J + c_R \cdot a] - (c_J + c_R \cdot a)$. Hence it is worth sending if and only if $p \geq \tau$, where $\tau = (c_J + c_R \cdot a) / ((L - c_R)J + c_R \cdot a)$. If $(L - c_R)J \leq c_J$, then $\tau \geq 1$ and the judge is never worth calling.

Proof. Not sending leaves expected cost $L \cdot p$. Sending costs $c_J + c_R(pJ + (1 - p)a) + L \cdot p(1 - J)$.

The difference is $V(p)$, which is linear and increasing in p . ■

Equivalently, a document should be judged if and only if the screen's likelihood ratio satisfies

$$\ell(s) \geq \ell^* = ((1 - \pi)/\pi) \cdot \tau / (1 - \tau).$$

Proposition 2 (depth). The set of documents worth judging, $\{s : \ell(s) \geq \ell^*\}$, expands as ℓ^* falls, and ℓ^* is decreasing in π . The cost-minimizing depth is therefore nondecreasing in the base rate π . It is also nondecreasing in J and L and nonincreasing in c_J and c_R .

Proof. τ is increasing in c_J ; τ falls with J and with L ; and $\partial\tau/\partial c_R = [a \cdot L \cdot J + c_J(J - a)] / D^2$, where D is the denominator of τ , which is positive when $J > a$. ℓ^* is increasing in τ and decreasing in π , and ℓ is nondecreasing in s . ■

Proposition 3 (blind spots). τ rises as the judge's detection rate J falls, and the judge is never worth calling when $J \leq c_J/(L - c_R)$. If the screen cannot tell generators apart, the relevant detection rate for a stream that mixes generators with weights w_g is the weighted mean $\bar{J} = \sum w_g \cdot J_g$, so a generator that the judge cannot detect lowers the judge's value for the whole stream.

Proposition 4 (precision at low base rates). Among the documents a judge flags in an unscreened stream, the share that is machine-written is $\pi J / (\pi J + (1 - \pi)a)$. It is low when π is small, even for a judge with a small false-alarm rate. Screening before judging raises the effective base rate among judged documents, which raises this share and reduces wasted investigations.

3.3 Strategic evasion

The cascade model treats authors as passive. Suppose instead that a machine-using author chooses an evasion effort e in $[0, 1]$ at cost $(\kappa/2)e^2$, which lowers the probability of detection to

$d(e)$, with $d(0) = d_0$ the rate against a non-adaptive author. The author gains B if the paper passes and loses P if it is detected, so $u(e) = B(1 - d(e)) - P \cdot d(e) - (\kappa/2)e^2$. Let $m = (B + P)/\kappa$ be the ratio of the stakes to the cost of effort. The author chooses e to minimize $m \cdot d(e) + e^2/2$, and the organization, anticipating this, cares about the detection rate $d(e^*)$ that remains in equilibrium.

Proposition 5 (evasion rises with the stakes). If d is differentiable, decreasing, and convex, the optimal effort e^* satisfies $m \cdot |d'(e^*)| = e^*$, or $e^* = 1$ at the corner. Effort is therefore nondecreasing in m and in the steepness of d , so equilibrium detection $d(e^*)$ falls as the stakes rise or evasion becomes cheaper.

Proposition 6 (the detectability paradox). If $d(e) = d_0(1 - e)$, then $e^* = \min(1, m \cdot d_0)$ and $d(e^*) = d_0(1 - m \cdot d_0)$ whenever $m \cdot d_0 \leq 1$. Equilibrium detection is maximized at $d_0 = 1/(2m)$, where it equals $1/(4m)$, and decreases in d_0 beyond that point. A screen that is very effective against non-adaptive authors provokes more evasion, and past a point it detects less than a weaker screen would.

Proof. The first-order condition gives $e = m \cdot d_0$. Substituting gives $d_0(1 - m \cdot d_0)$, whose derivative in d_0 is $1 - 2m \cdot d_0$. ■

Proposition 7 (robust design). Consider two screens built from different feature sets, with detection curves $d_A(e)$ and $d_B(e)$, where d_B is flat in e because its features cannot be edited cheaply. If $d_B(0) < d_A(0)$ but d_A falls steeply, there is a threshold $\hat{m}$ such that the screen built on B has higher equilibrium detection for every $m > \hat{m}$. An organization should prefer features that are costly to manipulate even at some cost in raw accuracy. This is the logic of costly signals (Spence 1973) applied to the screen's inputs.

3.4 Empirical implications

The model’s propositions guide the empirical work. We calibrate Propositions 1 to 4 with the measured detection behavior of the screen and the judge, and with the measured judge price; the costs c_R and L and the base rate π are not measurable from our data and are varied over illustrative values. We test Propositions 5 to 7 with an adversarial experiment that measures how the screen’s detection falls with evasion effort. Throughout, we report structural patterns and not estimates of any organization’s costs.

4. Data and Measurement

4.1 Human-authored papers

We sampled English-language papers published between 2015 and 2019, before large language models were widely used for scientific writing. Papers came from the open-access subset of PubMed Central (medicine; 300 sampled) and from arXiv (computer science; 200 sampled), drawn at random within each source. For PubMed Central we used the body paragraphs of the article XML. For arXiv we converted the PDFs to text, removed the reference list, and rejoined broken lines. We excluded documents shorter than 1,500 words, and the analysis requires at least four 500-token windows, which left 474 human papers.

4.2 Machine-generated papers

For each sampled paper we asked a generator to write a paper on the same topic, given the original title and abstract, using one of three prompts of increasing detail. We used three generators through a commercial API gateway: GPT-4o, Claude Sonnet 5.5, and Llama 3.3 70B Instruct, an open-weight model. A single request produced only about 800 words from GPT-4o, too short for window-based analysis, so each paper was written in four requests (introduction,

methods, results, and discussion with conclusion) and concatenated. Prompts were processed in a fixed random order, so each generator’s sample is random across fields.

GPT-4o produced 245 unique papers, with a median length of about 2,750 words. Claude produced 152 papers (median 4,566 words), with generation capped at that number to limit cost. Llama produced 202 unique papers (median about 2,970 words). Llama was added last, after the clustering model of Section 4.3 had been developed, and it serves as the confirmatory test. Because the generators wrote papers of different lengths, every document is truncated to the same length before analysis.

4.3 Events, windows, and the screen

Each document is cut into consecutive windows of 500 whitespace-delimited tokens, and only the first four windows (2,000 tokens) are kept for every document, human or machine, so that length cannot drive any result. In each window we count seven events: parentheses, square brackets, semicolons, paired quotation marks, a list of fourteen discourse markers (for example, *however*, *therefore*, *moreover*), passive constructions (a form of *be* plus a participle, found by pattern matching), and nominalizations (words ending in *-tion*, *-ment*, or *-ity*). The patterns for passives and nominalizations are approximate and misclassify some words. For each document and event we compute the mean count per window (the *rate*) and the dispersion index, and the baseline screen is a logistic regression on these fourteen quantities. A document-level Poisson test compares $(n - 1)$ times the dispersion index with a chi-squared distribution; with only four windows per document it has low power, so rejection rates understate departures from Poisson behavior.

A dispersion index at one window size says whether events cluster but not how. We therefore developed a scale-dependent clustering model. We treat the occurrences of each event as a

doubly stochastic Poisson process, or Cox process: events arrive as a Poisson process whose rate varies along the document. For such a process, the Fano factor in a window of w tokens is $F(w) = 1 + \text{Var}(\Lambda_w) / E(\Lambda_w)$, where Λ_w is the rate integrated over the window. If rate fluctuations are positively correlated over long ranges, as when citations gather in an introduction or passives in a methods section, $F(w)$ grows with w ; a pure Poisson process gives $F(w) = 1$ at every scale. We estimate $F(w)$ for each event at four window sizes (50, 100, 200, and 400 tokens) within the same 2,000 tokens, and summarize each event by its rate, the mean log Fano factor across scales, and the slope of $\log F(w)$ on $\log w$. Estimation is by moment matching and not a full likelihood fit; Fano factors are clipped to $[0.05, 20]$ and set to one when an event does not occur. This yields 21 features. The comparison with the baseline was fixed before the model was run, no setting was tuned on these data, and the model was designed after we had seen the baseline results, a limitation we return to in Section 7.

4.4 LLM judges

We asked two models, zero-shot and at temperature 0, to estimate for each excerpt the probability (0 to 100) that it was written by an AI language model: GPT-4o and Gemini 2.5 Pro. Both received one fixed prompt. They saw a random sample of 500 excerpts (200 human papers, of which 120 were from medicine and 80 from computer science, and 100 papers from each generator), in random order, with opaque identifiers and no labels. Each excerpt was the same first 2,000 tokens used for the screen, with markdown symbols and bare section titles removed from every excerpt to avoid trivial cues. Gemini 2.5 Pro reasons before answering, so we allowed it up to 8,000 output tokens; 12 of its 500 answers were unreadable. In the main analysis a document is excluded for every method if either judge’s answer is unreadable, which leaves 196 human papers and 92 to 95 papers per generator; as a sensitivity check we score unreadable

answers as 50. GPT-4o is also one of the generators, so its verdicts on GPT-4o papers involve self-recognition; Gemini is not a generator. We measured the price and latency of each judge.

4.5 Evaluation

Detection is evaluated by five-fold cross-validation in which all material from one source paper stays in one fold. We report the area under the ROC curve (AUC) with 95% bootstrap intervals over documents, and, where noted, the share of machine papers detected when 5% of human papers are flagged. To study transfer, a screen is trained on human papers and the other generators and tested on a generator it never saw; this “unseen” protocol is also what makes a screen comparable to a zero-shot judge. Three analyses were specified before they were run: the test of the clustering model on Llama, the comparison of screens with judges and a rank-average fusion of the screen and the Gemini judge, and the cascade calibration. The analysis code, prompts, and generated papers will be made available in a public repository upon acceptance, together with the script that re-collects the human papers.

5. Results

5.1 Human text is not Poisson

Table 1 reports, for each event, the median dispersion index and the share of documents whose counts reject the Poisson null in the direction of over-dispersion. Only discourse markers behave roughly as the Poisson model predicts in human text, with a median index of 1.00 and a rejection rate of 6%, close to the 5% expected by chance. Punctuation is clearly over-dispersed: 49% of documents reject the null for square brackets, 42% for parentheses, and 36% for semicolons. Citations and formulas plausibly gather in particular parts of a paper, and this is the kind of clustering that a Poisson model cannot represent. A single distance from a Poisson process

therefore has almost no discriminating power, with an AUC of 0.54 for GPT-4o, 0.51 for Claude, and 0.53 for Llama.

Table 1. Median dispersion index and percentage of documents rejecting the Poisson null toward over-dispersion (5% level), by source. Index = variance / mean; 1 indicates Poisson behavior.

Event	Human: index (% over)	GPT-4o: index (% over)	Claude: index (% over)	Llama: index (% over)
Square brackets	2.53 (48.8)	1.33 (36.4)	1.22 (22.0)	2.00 (38.5)
Parentheses	2.22 (42.2)	1.78 (28.6)	2.21 (40.1)	2.02 (39.6)
Semicolons	2.00 (36.2)	1.00 (12.4)	2.00 (36.0)	2.00 (35.0)
Nominalizations	1.67 (27.4)	0.76 (7.3)	1.10 (15.1)	1.27 (21.8)
Quotation marks	1.30 (24.4)	3.00 (60.0)	2.00 (37.7)	3.00 (55.3)
Passive constructions	1.20 (18.8)	2.51 (47.3)	1.79 (37.5)	2.02 (35.1)
Discourse markers	1.00 (6.2)	1.11 (7.8)	1.22 (4.6)	1.11 (9.4)

Machine text departs from human text in both directions, which argues against the tempting story that it is uniformly too regular. Nominalizations are less dispersed in all three generators (medians of 0.76, 1.10, and 1.27, against 1.67 in human text). Passive constructions and quotation marks are more dispersed in all three (passives 2.51, 1.79, and 2.02 against 1.20; quotation marks 3.00, 2.00, and 3.00 against 1.30). That these directions agree across generators makes them more credible as features of machine writing than as quirks of one model. One caution applies to the passive result. Because each paper was generated section by section and the four retained windows roughly follow those sections, elevated passive dispersion may partly reflect a procedural artifact. Figure 1 shows the distributions for human and GPT-4o text.

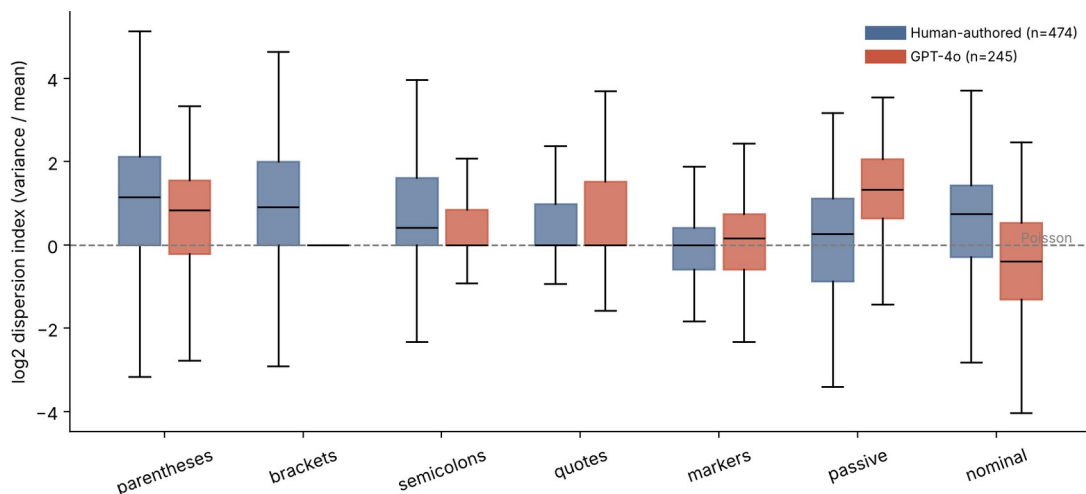


Figure 1. Distribution of log2 dispersion indices by event for human-authored and GPT-4o papers. Dashed line: Poisson (index of 1). Square brackets for GPT-4o are degenerate because most machine papers contain none (index set to one by convention).

5.2 Screening within and across generators

With the fourteen baseline features as separate predictors, cross-validated logistic regression reaches an AUC of 0.987 for GPT-4o, 0.913 for Claude, and 0.962 for Llama when it is trained and tested on human papers plus one generator. Event rates carry most of this signal, and Table 2 shows why they are specific to each generator. GPT-4o uses far fewer square brackets, semicolons, and quotation marks than human authors and more nominalizations and discourse markers. Claude uses *more* semicolons than human authors (3.54 per 500 tokens, against 1.95) and fewer discourse markers (1.08 against 1.84). Llama shows a third pattern, with many parentheses and few brackets. Because semicolons fall below the human rate for one generator and rise above it for another, no rule based on average frequency describes machine text in general.

Table 2. Mean event count per 500-token window, by source.

Event	Human	GPT-4o	Claude	Llama
Parentheses	9.80	7.39	11.39	14.66
Square brackets	4.46	0.21	0.41	0.17
Semicolons	1.95	0.51	3.54	1.17
Quotation marks	0.95	0.53	1.23	0.55
Discourse	1.84	2.38	1.08	2.12

Event	Human	GPT-4o	Claude	Llama
markers				
Passive constructions	8.08	6.18	8.61	6.51
Nominalizations	24.43	34.08	27.34	26.67

Performance falls sharply when the screen meets a generator it was not trained on (Table 3). A GPT-4o-trained screen reaches an AUC of 0.73 on Claude papers (95% interval 0.69 to 0.78), down from 0.99 on GPT-4o itself, and a Claude-trained screen reaches 0.81 on GPT-4o papers (0.78 to 0.84). Rates transfer worst (0.66 and 0.73), whereas dispersion transfers better (0.70 and 0.81), consistent with the sign pattern in Table 2: differences in rates reverse across generators, while the differences in dispersion for passives, quotation marks, and nominalizations point the same way. A screen trained on human papers alone, which would not need generator examples, performs near chance (AUCs of 0.45 to 0.66 across three methods; Appendix Table A1). A screen is therefore only as good as the examples it has seen.

Table 3. AUC (95% bootstrap interval) by training and test generator and feature family. Training and test folds are grouped by source paper; test documents include held-out human papers.

Train → Test	Rates	Dispersion	Both
GPT-4o → GPT-4o	0.971 [0.96, 0.98]	0.856 [0.83, 0.88]	0.987 [0.98, 0.99]
Claude → Claude	0.904 [0.88, 0.93]	0.751 [0.71, 0.80]	0.913 [0.89, 0.94]
GPT-4o → Claude	0.659 [0.62, 0.70]	0.704 [0.65, 0.75]	0.730 [0.69, 0.78]
Claude → GPT-4o	0.728 [0.69, 0.76]	0.808 [0.77, 0.84]	0.810 [0.78, 0.84]

5.3 Clustering features

The scale-dependent clustering features improve detection modestly and consistently. On the GPT-4o and Claude data on which the model was developed, they help in seven of eight paired comparisons, with the largest gain for dispersion-only detection within a generator (Appendix Table A2). The confirmatory test uses Llama papers, which played no part in developing the model, with an analysis specified before they were generated. Trained on human papers plus GPT-4o and Claude pooled, the baseline features detect Llama papers with an AUC of 0.886 (95% interval 0.86 to 0.91), and the clustering features reach 0.932 (0.91 to 0.95), a difference of

+0.046 (0.030 to 0.063; Table 4). The improvement also appears when training on either generator alone. The clustering features do not rescue human-only detection, which stays near chance except for a modest 0.65 on Llama.

Table 4. Confirmatory test: detection of Llama 3.3 70B papers by screens that never saw Llama. Baseline = rates plus the 500-token dispersion index; Clustering = rates plus scale-dependent clustering features.

Trained on (plus human papers)	Baseline AUC	Clustering AUC	Difference [95% CI]
GPT-4o and Claude (primary)	0.886 [0.86, 0.91]	0.932 [0.91, 0.95]	+0.046 [+0.030, +0.063]
GPT-4o only	0.913 [0.89, 0.93]	0.953 [0.94, 0.97]	+0.041 [+0.027, +0.054]
Claude only	0.737 [0.70, 0.77]	0.796 [0.76, 0.83]	+0.058 [+0.034, +0.084]

5.4 LLM judges and their fusion with the screen

Table 5 gives the AUC of each method for separating human papers from each generator’s papers on the judged excerpts. The two judges differ greatly. GPT-4o as judge is a modest detector of GPT-4o papers (AUC 0.76) and Llama papers (0.85) and worse than chance on Claude papers (0.45). At a threshold of 50 it flags 24% of human papers as AI-generated, against 59% of GPT-4o papers, 85% of Llama papers, and only 11% of Claude papers. Gemini 2.5 Pro is much stronger on two generators, with AUCs of 0.97 for both GPT-4o and Llama papers, and it flags only 2% of human papers while flagging 86% of GPT-4o and 89% of Llama papers. On Claude papers, however, it reaches only 0.66 and flags 1% of them, fewer than the 2% of human papers it flags.

Table 5. AUC (95% bootstrap interval) on the same documents. “Unseen” means trained on human papers plus the other two generators, never on the target. Sample sizes are human / machine documents. Scoring the unreadable Gemini answers as 50 instead of excluding them changes the Gemini AUCs to 0.970, 0.653, and 0.973.

Target generator	GPT-4o judge	Gemini 2.5 Pro judge	Screen, baseline features (unseen)	Screen, clustering features (unseen)
GPT-4o (n = 196 / 92)	0.760 [0.70, 0.81]	0.972 [0.95, 0.99]	0.927 [0.90, 0.95]	0.946 [0.92, 0.97]
Claude (n = 196 / 94)	0.450 [0.39, 0.51]	0.656 [0.59, 0.72]	0.759 [0.70, 0.82]	0.775 [0.72, 0.83]
Llama (n = 196 / 95)	0.846 [0.80, 0.89]	0.974 [0.95, 0.99]	0.884 [0.85, 0.92]	0.933 [0.90, 0.96]

How the screen compares with a judge therefore depends on the judge. Against GPT-4o, the clustering screen is better on all three generators (paired AUC differences of +0.19 for GPT-4o, 95% interval 0.13 to 0.25; +0.33 for Claude, 0.24 to 0.40; and +0.09 for Llama, 0.04 to 0.14). Against Gemini 2.5 Pro the comparison reverses for two generators and holds for the third: the screen is lower on Llama papers (−0.04, interval −0.08 to −0.01), lower on GPT-4o papers by an amount whose interval reaches zero (−0.03, interval −0.06 to +0.01), and higher on Claude papers (+0.12, interval 0.04 to 0.20). Medicine papers alone give similar results, so extraction artifacts in the arXiv text do not explain the pattern. The comparison carries an asymmetry: the screen uses labeled examples from other generators and the judges use none.

The two methods fail in different places, and a rank average of the screen and the Gemini judge, specified in advance, exploits that (Table 6). It reaches an AUC of 0.994 on GPT-4o papers and 0.989 on Llama papers, better than either component alone, with gains over Gemini of +0.022 (95% interval 0.008 to 0.040) and +0.014 (0.001 to 0.031). On Claude papers the fusion reaches 0.790, far above the judge but not distinguishable from the screen alone (+0.015, interval −0.028 to 0.059). The share detected at a 5% false-positive rate shows the practical effect: 98% of GPT-4o papers against 87% for the judge and 64% for the screen, 96% of Llama papers against 89% and 73%, and 39% of Claude papers against 17% for the screen and 1% for the judge.

Table 6. AUC (95% bootstrap interval) of the rank-average fusion and its components on the same documents as Table 5, and the share of machine papers detected at a 5% false-positive rate. The 5% thresholds are set on the same documents and are descriptive, not estimates of deployed performance.

Target generator	Screen (clustering)	Gemini judge	Fusion	Detected at 5% FPR: screen / Gemini / fusion
GPT-4o	0.946 [0.92, 0.97]	0.972 [0.95, 0.99]	0.994 [0.99, 1.00]	0.64 / 0.87 / 0.98
Claude	0.775 [0.72, 0.83]	0.656 [0.59, 0.72]	0.790 [0.73, 0.84]	0.17 / 0.01 / 0.39
Llama	0.933 [0.90, 0.96]	0.974 [0.95, 0.99]	0.989 [0.98, 1.00]	0.73 / 0.89 / 0.96

5.5 Calibrating the cascade model

We now use the measured behavior of the screen and the Gemini judge to calibrate the model of Section 3. The judge’s price is US\$0.0174 per document (measured), and at a threshold of 50 it flags 2% of human papers ($\alpha = 0.02$) and detects 86% of GPT-4o papers, 89% of Llama papers, and 1% of Claude papers. Table 7 gives the screen’s recall at several depths, where depth is the share of human papers that the screen sends to the judge. The mixture row averages the three generators with equal weight, giving a mixture detection rate $\bar{J}$ of 0.59.

Table 7. Share of machine papers sent to the judge, and sent and flagged, when the screen passes the stated share of human papers (empirical rates; no independence assumed).

Target	5%: sent / sent and flagged	20%	50%	100%
GPT-4o	0.64 / 0.55	0.98 / 0.85	1.00 / 0.87	1.00 / 0.87
Claude	0.17 / 0.00	0.61 / 0.00	0.86 / 0.00	1.00 / 0.01
Llama	0.73 / 0.66	0.87 / 0.80	0.99 / 0.89	1.00 / 0.89
Mixture	0.51 / 0.41	0.82 / 0.55	0.95 / 0.59	1.00 / 0.59

The costs of reviewing a flagged paper and of a missed machine paper cannot be measured from our data, so we use illustrative values: US\$20 per flagged paper and losses of US\$100 and US\$500 per missed paper, with the judge priced at the measured US\$0.0174 and at 100 times that figure (for example, a longer input or a reasoning model with a long output). Using the empirical rates and no independence assumption, Table 8 gives the expected cost per 1,000 documents of four policies: no screening, review everything above the screen threshold, judge every document, and the best cascade.

Table 8. Expected cost per 1,000 documents (US\$) for the equal-weight mixture of generators, with review at US\$20 per flagged paper (assumed). q^* is the share of all documents sent to the judge at the cascade’s optimum.

L	Judge price	Base rate	Best cascade q^*	No screening	Screen only	Judge on all	Cascade	Saving vs judge-all
US\$100	×1	1%	14%	1,000	973	948	594	37%
US\$100	×1	10%	41%	10,000	6,225	5,652	5,420	4%
US\$100	×1	30%	67%	30,000	12,647	16,104	15,955	1%
US\$100	×100	1%	5%	1,000	973	2,671	767	71%
US\$100	×100	10%	24%	10,000	6,225	7,374	6,051	18%

L	Judge price	Base rate	Best cascade q*	No screening	Screen only	Judge on all	Cascade	Saving vs judge-all
US\$100	×100	30%	56%	30,000	12,647	17,826	16,959	5%
US\$500	×1	1%	36%	5,000	3,465	2,582	2,284	12%
US\$500	×1	10%	59%	50,000	12,092	21,986	21,795	1%
US\$500	×100	1%	18%	5,000	3,465	4,304	2,713	37%
US\$500	×100	10%	45%	50,000	12,092	23,708	22,684	4%

Five patterns emerge, and they hold across the assumptions we tried.

1. **Depth rises with the base rate**, as Proposition 2 requires. For the mixture at L = US\$100, the optimal share judged goes from 14% at a 1% base rate to 41% at 10% and 67% at 30% (Figure 2B).
2. **The judge's price is a minor part of total cost; its false alarms are not.** For the mixture at a 1% base rate, judging everything costs US\$17 in judge calls, US\$522 in investigations, and US\$408 in missed papers per 1,000 documents. The cascade cuts investigations to US\$102 by judging only 14% of documents, at the price of a small rise in missed papers (US\$408 to US\$490). The saving comes from avoiding false alarms, in line with Proposition 4: at a 1% base rate and a 2% false-alarm rate, only about 23% of the papers an unscreened judge flags are machine-written (77% at 10%, 93% at 30%).
3. **Savings are largest where machine papers are rare and the judge is expensive**, up to 71% of cost at a 1% base rate with a judge 100 times pricier, and small (1% to 5%) at a 30% base rate.
4. **At high base rates, reviewing a deep slice of the screen's output beats any use of the judge** under these assumed costs. This reflects the assumption that review is cheap and settles a paper perfectly, and it is not a recommendation to review everything.
5. **The judge's blind spot makes it worth little for Claude papers.** Claude's detection rate of 0.01 gives $\tau = 0.35$ at L = US\$100, so the judge is worth calling only when the

screen’s likelihood ratio is at least 53 at a 1% base rate; in the data, the judge flagged none of the Claude papers the screen sent it. For the mixture, $\bar{J} = 0.59$ raises τ to 0.009, against 0.006 for GPT-4o alone (Proposition 3).

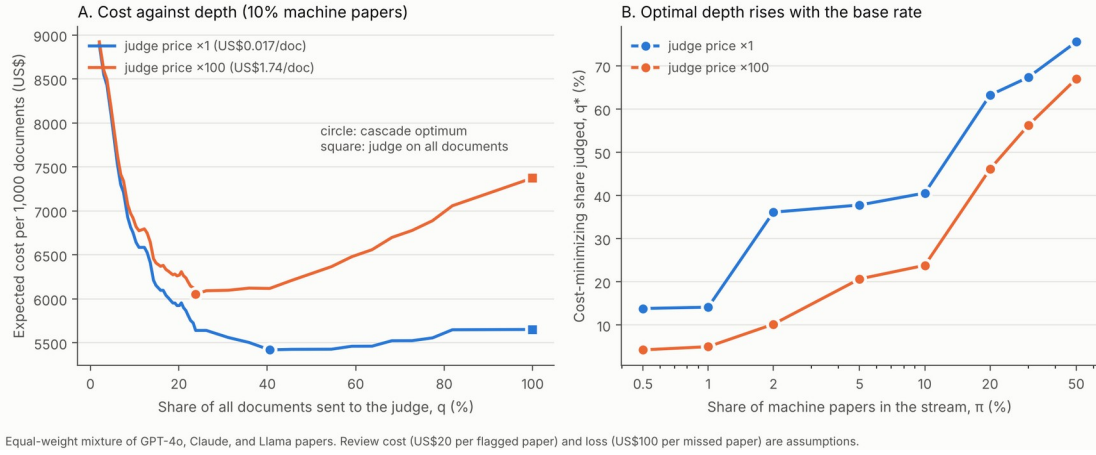


Figure 2. (A) Expected cost per 1,000 documents against the share of documents sent to the judge, with 10% machine papers, a loss of US\$100 per missed paper, and review at US\$20 per flagged paper. (B) Cost-minimizing share judged against the base rate. Both panels use the mixture of generators.

5.6 Strategic evasion

We tested Propositions 5 to 7 with a white-box attack on the five events that are easy to edit: parentheses, square brackets, semicolons, quotation marks, and discourse markers. For each machine paper, the attack moves a share e of the way from the paper’s own count to the human mean count for each event, and places the edits in two of the four windows, as human text clusters. Passives and nominalizations are not edited, so the attack is a lower bound on what an adaptive author could do. It knows the features but not the detector’s weights. The detector (clustering features) is trained on clean text from human papers and the other two generators, and attacked texts are only scored. The edits are programmatic and crude, so the edited text is unnatural.

Table 9. AUC / share of machine papers detected at a 5% false-positive rate, by evasion effort e .

Target	$e = 0$	$e = 0.25$	$e = 0.5$	$e = 0.75$	$e = 1$
GPT-4o	0.959 / 0.73	0.888 / 0.33	0.849 / 0.24	0.824 / 0.12	0.800 / 0.10
Claude	0.777 / 0.18	0.634 / 0.05	0.539 / 0.02	0.473 / 0.01	0.471 / 0.01

Target	$e = 0$	$e = 0.25$	$e = 0.5$	$e = 0.75$	$e = 1$
Llama	0.932 / 0.69	0.852 / 0.37	0.799 / 0.20	0.763 / 0.17	0.721 / 0.12
Mixture	0.889 / 0.53	0.791 / 0.25	0.729 / 0.15	0.687 / 0.10	0.664 / 0.08

The screen degrades quickly (Table 9). Averaged over generators, the AUC falls from 0.889 to 0.664, and the share detected at a 5% false-positive rate falls from 53% to 8% under full attack. Half of the loss occurs at $e = 0.25$, so the first edits are the most effective. This makes $d(e)$ steep near zero and, by Proposition 5, pushes equilibrium effort up. For Claude papers the AUC falls below 0.5, so edited Claude papers look *more* human than real human papers. Table 10 separates the features. A screen built only on the two unedited events (passives and nominalizations) is unaffected by the attack, with a mixture AUC of 0.780 before and 0.779 after, whereas a screen built only on the five edited events falls from 0.849 to 0.491, which is chance. The robust features are weaker without an attack (0.780 against 0.889 for all seven events).

Table 10. Detection by feature set under no attack and full attack, mixture of generators.

Feature set	Target	$e = 0$: AUC / detected	$e = 1$: AUC / detected
Passive and nominalization only	Mixture	0.780 / 0.34	0.779 / 0.33
Five edited events only	Mixture	0.849 / 0.35	0.491 / 0.00
All seven events	Mixture	0.889 / 0.53	0.664 / 0.08

Table 11 puts the measured detection curve (53% falling to 8%) into the author’s problem for a range of m , and compares the full screen with a screen built on the hard-to-edit events, whose detection is nearly flat (34% to 33%). Once evasion is not very costly relative to the stakes, the full screen’s equilibrium detection falls below that of the hard-to-edit screen; the crossover is at $m \approx 0.16$ (Proposition 7). At $m = 1$, the full screen is expected to detect 19% of machine papers, against 34% for the hard-to-edit design, although the full screen detects 53% of papers from non-adaptive authors. This is the detectability paradox in numbers (Proposition 6).

Table 11. Equilibrium evasion effort and the detection rate that remains (share detected at a 5% false-positive rate), by the ratio of the stakes to the cost of evasion.

$m = (B + P)/\kappa$	Full screen: e^*	Full screen: detection	Hard-to-edit only: e^*	Hard-to-edit only: detection
0.10	0.11	0.41	0.00	0.34
0.25	0.25	0.25	0.01	0.34
0.50	0.25	0.25	0.02	0.34
1.00	0.41	0.19	0.04	0.34
2.00	0.50	0.15	0.08	0.34
4.00	0.75	0.10	0.16	0.33

We also tested a generic attack that needs no knowledge of the features. A language model, Llama 3.3 70B Instruct, rewrote each machine paper in the judge sample (300 papers, 100 per generator) after being asked to make it read as if written by an experienced human researcher; the papers were rewritten in chunks of 1,000 tokens with the target length stated, at a cost of about US\$0.003 per paper. Because the attacker is itself one of the generators, a screen trained on “other generators” has seen its style when the target is GPT-4o or Claude, so Table 12 reports both the standard protocol and one that excludes Llama from training. The rewrite did not evade the screen. Averaged over generators, the AUC rose from 0.885 to 0.914 under the standard protocol and from 0.837 to 0.884 with Llama excluded, and the share detected at a 5% false-positive rate stayed at about 0.53 or rose from 0.37 to 0.42. For Claude papers the rewrite made detection clearly easier (AUC 0.750 to 0.905 with Llama excluded); for GPT-4o papers it changed little (0.830 to 0.813).

Table 12. Effect of a generic LLM rewrite on detection by the clustering screen: AUC / share of machine papers detected at a 5% false-positive rate, on the same papers before and after the rewrite. The last two columns exclude Llama, the attacker, from the screen’s training. For Llama papers the attacker and the target coincide, so the two protocols are identical.

Target	Original: AUC / detected	Rewritten: AUC / detected	Original (Llama excluded)	Rewritten (Llama excluded)
GPT-4o	0.950 / 0.71	0.924 / 0.54	0.830 / 0.20	0.813 / 0.11
Claude	0.773 / 0.22	0.884 / 0.42	0.750 / 0.21	0.905 / 0.51
Llama	0.932 / 0.70	0.935 / 0.63	0.932 / 0.70	0.935 / 0.63
Mixture	0.885 / 0.54	0.914 / 0.53	0.837 / 0.37	0.884 / 0.42

Table 13 shows why. The rewrite *removed* parentheses, bracketed citations, semicolons, and passives, moving counts further from the human mean and not toward it. A generic paraphrase imposes the paraphraser’s habits, and these are not the habits of human scientific writers.

Table 13. Mean event count per 500-token window before and after the LLM rewrite, for four events.

Event	Human	GPT-4o: original → rewritten	Claude: original → rewritten	Llama: original → rewritten
Parentheses	9.80	7.33 → 5.00	11.38 → 5.72	14.79 → 7.38
Square brackets	4.46	0.23 → 0.06	0.50 → 0.15	0.12 → 0.02
Semicolons	1.95	0.53 → 0.18	3.54 → 1.53	1.26 → 0.56
Passive constructions	8.08	6.31 → 5.38	8.71 → 6.76	6.47 → 5.57

The two attacks together say that what matters for evasion is knowledge of the screen’s features, not rewriting power as such. An author who knows which events are counted can steer them, as the programmatic edit does, whereas an author who simply asks a model to sound human cannot. In the terms of the model, the generic rewrite is effort without effect, so $d(e)$ does not fall and a rational author has no reason to use it, while the feature-aware edit has a steep $d(e)$. A hybrid attack combines the two: the language model rewrites the paper, and the feature-aware edit is then applied to the rewritten text. It was no stronger than the edit alone (Table 14). Under full strength the mixture AUC was 0.681 under the standard protocol and 0.688 with Llama excluded, against 0.664 and 0.675 for the edit on the original papers, and the share detected at a 5% false-positive rate was 7% and 9% against 9% and 10%. The rewrite adds nothing beyond the edit: it helps slightly for GPT-4o papers (AUC 0.759 against 0.784 under the standard protocol) and hurts for Claude papers (0.561 against 0.481), because it moves their punctuation toward the paraphraser’s habits and so makes them easier to detect. A hybrid at half strength ($e = 0.5$) kept 12% to 13% of papers detectable. A residual of detection (AUC about 0.68) survives even the full hybrid, which comes from the events the attack leaves untouched, passives and nominalizations, and from the clustering structure of the others.

Table 14. Hybrid attack on the same 300 machine papers, with Llama (the rewriter) excluded from the screen’s training: AUC / share detected at a 5% false-positive rate. Under the standard protocol the mixture results are 0.885 / 0.54 (original), 0.914 / 0.53 (rewrite only), 0.664 / 0.09 (edit only), 0.742 / 0.12 (hybrid at $e = 0.5$), and 0.681 / 0.07 (hybrid at $e = 1$).

Condition	GPT-4o	Claude	Llama	Mixture
Original (no attack)	0.830 / 0.20	0.750 / 0.21	0.932 / 0.70	0.837 / 0.37
Rewrite only	0.813 / 0.11	0.905 / 0.51	0.935 / 0.63	0.884 / 0.42
Feature-aware edit only ($e = 1$)	0.804 / 0.13	0.495 / 0.03	0.727 / 0.13	0.675 / 0.10
Rewrite, then edit ($e = 0.5$)	0.770 / 0.10	0.674 / 0.09	0.782 / 0.21	0.742 / 0.13
Rewrite, then edit ($e = 1$)	0.800 / 0.10	0.541 / 0.03	0.723 / 0.13	0.688 / 0.09

None of the attackers is the strongest conceivable. The edit leaves passives and nominalizations alone, and none of the attacks optimizes against the detector’s weights.

We then gave the Gemini judge two sets of attacked papers: the hybrid-attacked papers (rewritten and then edited) and the edit-only papers, each the 300 machine papers in their first 2,000 tokens (US\$11.27 in judge calls for the two sets). The judge was not fooled by either (Table 15). For GPT-4o and Llama papers its AUC did not fall: the paired differences in AUC on the same papers were +0.004 (95% interval -0.017 to $+0.024$) and +0.013 (-0.003 to $+0.033$) for the hybrid attack, and +0.008 (-0.009 to $+0.029$) and +0.014 ($+0.002$ to $+0.032$) for the edit alone. For Claude papers both attacks made detection *easier*, by +0.167 (0.102 to 0.228) for the hybrid and +0.080 (0.021 to 0.144) for the edit alone, with the share detected at a 5% false-positive rate rising from 0.01 to 0.35 and 0.05. The screen, on the same papers, fell to an AUC of 0.536 and 0.493 for Claude papers and to 0.724 and 0.721 for Llama papers. Roughly half of the judge’s improvement for Claude comes from the edit and the rest from the rewrite, whose style the judge may recognize as machine-like; we did not give the judge rewrite-only papers, and we did not test why the edit helps the judge, although inserted citation numbers in mid-sentence may look unnatural. What the results do show is that attacks that defeat the count-based screen do not defeat this judge.

Table 15. The screen, the Gemini judge, and their rank-average fusion before and after the edit-only and hybrid attacks: AUC / share of machine papers detected at a 5% false-positive rate. The screen here is trained without the target generator and without Llama (the rewriter), so its original-text values differ from those in Tables 5 and 6, which include Llama in training.

Target	Condition	Screen: AUC / detected	Gemini judge: AUC / detected	Fusion: AUC / detected
GPT-4o	Original	0.817 / 0.12	0.972 / 0.87	0.966 / 0.82
GPT-4o	Edit only	0.791 / 0.08	0.980 / 0.85	0.962 / 0.82
GPT-4o	Hybrid attack	0.790 / 0.07	0.975 / 0.87	0.958 / 0.77
Claude	Original	0.746 / 0.22	0.656 / 0.01	0.774 / 0.35
Claude	Edit only	0.493 / 0.03	0.734 / 0.05	0.664 / 0.11
Claude	Hybrid attack	0.536 / 0.03	0.840 / 0.35	0.760 / 0.26
Llama	Original	0.929 / 0.73	0.974 / 0.89	0.989 / 0.96
Llama	Edit only	0.721 / 0.15	0.986 / 0.95	0.943 / 0.66
Llama	Hybrid attack	0.724 / 0.15	0.985 / 0.90	0.937 / 0.65

Because the fusion averages in a screen that the attack has made uninformative, it did worse under attack than the judge alone (for example 0.937 against 0.985 for Llama papers). The cascade, which uses the screen as a gate, suffers in the same way, because attacked papers fall out of the top-scoring share and never reach the judge (Table 16). With machine papers at 10% of the pool, sending 30% of documents to the judge detected 88% of Llama papers before the attack and 54% after it; the cost saving of the cascade is therefore bought with a dependence on a screen that an informed author can defeat. Judging every document was unaffected (0.96 to 0.94 for Llama papers).

Table 16. Cascade before and after the hybrid attack: share of machine papers detected when at most 5% of human papers are flagged, in pools with 10% machine papers (95% intervals over bootstrap pools). The screen is trained without the target generator and without Llama.

Target	Share judged (q)	Original	Hybrid attack
GPT-4o	30%	0.76 [0.59, 0.91]	0.70 [0.48, 0.91]
GPT-4o	50%	0.90 [0.73, 1.00]	0.89 [0.73, 1.00]
GPT-4o	100%	0.93 [0.77, 1.00]	0.93 [0.77, 1.00]
Claude	30%	0.30 [0.05, 0.55]	0.20 [0.05, 0.36]
Claude	50%	0.12 [0.00, 0.34]	0.31 [0.09, 0.52]
Claude	100%	0.11 [0.00, 0.27]	0.41 [0.18, 0.64]
Llama	30%	0.88 [0.73, 1.00]	0.54 [0.32, 0.73]
Llama	50%	0.96 [0.86, 1.00]	0.78 [0.59, 0.91]
Llama	100%	0.96 [0.86, 1.00]	0.94 [0.82, 1.00]

5.7 Cost

Computing all events and the baseline indices took 4 to 6 milliseconds per document on one CPU core, roughly 160 to 230 documents per second depending on machine load, with no external service calls, so the marginal monetary cost is zero. The judges cost about US\$0.0073 and 1.7 seconds per excerpt for GPT-4o (US\$3.63 for 500 excerpts) and about US\$0.0174 and 23.4 seconds per excerpt for Gemini 2.5 Pro (US\$8.68), a figure that includes the reasoning tokens it produced. Screening 1,000 documents would thus cost about US\$7 and half an hour of sequential calls with GPT-4o, and about US\$17 and more than six hours with Gemini. For context, producing one machine paper in four requests cost about US\$0.04 for GPT-4o, US\$0.13 for Claude, and US\$0.002 for Llama through the gateway we used.

6. Discussion

The central lesson is that detection accuracy is the wrong quantity to optimize in isolation. For an organization, the value of a screen depends on the base rate, on the cost of false alarms and misses, and on whether authors respond. Each of these shifts the answer in ways that headline accuracy hides. Our model identifies how, and the calibration shows that the effects are large.

First, the base rate and the judge's false alarms matter more than the judge's price. A judge with a 2% false-alarm rate produces mostly false alarms when machine papers are rare, and the screen's economic contribution is to raise the base rate the judge sees. This is why a cascade saves money at low base rates, and why the saving comes from fewer investigations and not from lower judge fees. Second, blind spots dominate. The strong judge was excellent on two generators and almost useless on Claude, and the weaker judge was wrong in the wrong direction on Claude. A screening organization cannot know which kind of generator it faces, so the relevant detection rate is a weighted average that a single blind spot drags down. The cheap

screen, by contrast, was weaker than the strong judge on two generators and stronger on the third, at a cost orders of magnitude lower, and a simple rank average of the two did better than either alone.

Third, a screen whose features are known is fragile. A crude edit of five easily changed events cut detection from 53% to 8%, whereas a generic LLM rewrite did not evade the screen and for one generator made detection easier, so the fragility lies in knowing the features and not in rewriting power as such. Publishing the features, as research does, supplies that knowledge. The model shows why the edit result should be expected: when evasion is cheap relative to the stakes, equilibrium detection is far below the figure that non-adaptive authors would produce. The detectability paradox means that a very effective screen can provoke enough evasion to underperform a weaker one. Designing a screen therefore means choosing features that are costly to manipulate, not only features that discriminate. In our data, passives and nominalizations played that role, which echoes the economic logic that a signal is informative only to the extent that it is costly to fake (Spence 1973; Connelly et al. 2011). Because we publish our method, the relevant benchmark for any deployed version is the equilibrium detection rate, not the headline accuracy in this paper. The judge, by contrast, was not fooled by the attacks that defeated the screen, neither the edit alone nor the hybrid, which suggests that the two stages of a cascade differ in robustness as well as in price. A cascade that gates the judge with a known, editable screen inherits the screen’s fragility, and one that gates it with features that are costly to edit would not.

Fourth, accuracy does not transfer across generators, and a screen that needs no examples of machine text does not work in our setting. Both findings point to maintenance as the dominant cost of any screen: it must be recalibrated as generators change and as authors adapt. We

therefore see these screens as a low-cost first filter whose output is routed to a judge or to human review, and not as grounds for action on their own. The explainability of count-based features can help an editor discuss a flag with an author, but we have not tested whether it improves reviewers' decisions.

7. Limitations and Future Research

The analysis rests on three generators and one genre, English-language scientific papers, so the results may not carry over to other models, languages, or text types. The sharp drop in transfer between GPT-4o and Claude is itself a warning that future releases may differ. Samples are modest: the Claude sample (152) is smaller than the GPT-4o (245) and Llama (202) samples because we capped generation to limit cost, bootstrap intervals for cross-generator AUCs are about ± 0.05 , and the judged subset of 500 documents is small enough that the cascade and fusion results should be read as indicative. The confirmatory test of the clustering model uses one held-out generator. The model was designed after we had seen baseline results, and the development comparisons involve several paired tests without correction for multiplicity.

Several features of the data construction could affect the results. We generated papers in four requests to reach analyzable length, which may introduce structural artifacts such as inflated passive dispersion and is not how most users would produce a paper. Human papers come from two extraction pipelines while machine papers are plain text, although medicine-only results suggest that extraction artifacts do not drive the judge comparison. Our prompts asked for in-text citations and a given structure, so rate differences such as those in square brackets may reflect these instructions. Four windows per document give low power, and passive and nominalization detection rely on patterns, not a parser.

The economic model is simple. It assumes that investigating a flagged paper settles its status at a fixed cost, which is generous given that human reviewers often cannot distinguish machine text, and that capacity is unlimited. The costs of review and of a missed paper and the base rate are assumed, not measured, so the cost results are structural illustrations and not estimates for any organization. The judge’s operating point is fixed at a threshold of 50. The strategic extension considers a single decision by a single author, takes the base rate as given, and uses a quadratic cost of effort. The programmatic adversarial test is crude; it is a lower bound on an adaptive author, who could edit passives and nominalizations too, or optimize against the detector’s weights. The generic LLM rewrite used a single attacker (Llama), a single prompt, and the clustering screen only. The hybrid of rewriting and feature-aware editing was no stronger than the edit alone, but neither attack edits passives and nominalizations or optimizes against the detector’s weights, so an adaptive author could do better than any of our attacks. The hard-to-edit feature set was chosen because our attack left it untouched, so its robustness is conditional on that attack. The judge was evaluated on edit-only and hybrid-attacked papers but not on rewrite-only papers, with one attacker and the Gemini judge only; the GPT-4o judge was not evaluated on attacked papers. The judge saw edits that an adaptive author could conceal better, and a human reader might notice them. We did not run MAUVE or BERTScore, we did not test light human editing of machine text, and we did not test whether explainable evidence improves human decisions.

Four steps would turn this pilot into a fuller test. The first is a stronger adversary: an attack that also edits passives and nominalizations or optimizes against the detector’s weights, evaluated against both the screen and the judges, with rewrite-only papers also given to the judge, and with more than one attacker model. The second is to measure the parameters the model needs, with

data on the base rate of machine-written submissions, on the cost of investigating a flag, and on the accuracy of human review, which would replace our illustrative values. The third is to add generators, especially open-weight models and future releases, and to replace the Poisson benchmark with a negative binomial reference and the moment estimator with a full likelihood. The fourth is a randomized experiment in which domain experts classify real and fabricated papers with and without an explainable indicator, which would test whether the transparency of count-based features improves decisions.

8. Conclusion

Whether to screen for machine-written scholarly text is an economic decision, and the value of a screen depends on three things that detection accuracy leaves out: the base rate of machine papers, the cost of false alarms and misses, and how authors respond. In a model of cascaded screening, a free statistical screen is worth most because it raises the base rate that an expensive judge sees, a judge with a blind spot is worth little, and a screen whose features are known can lose most of its power through evasion. Calibrated on three generators, the cheap screen detected machine papers well only within a generator, a strong LLM judge missed one generator almost completely, a simple cascade kept most of the judge’s detection at a fraction of its cost when machine papers were rare, and a crude feature-aware edit cut the screen’s detection from 53% to 8%, whereas a generic LLM rewrite did not evade it. The same edit, alone or after a rewrite, did not fool the judge, and the cascade, which depends on the screen, lost much of its detection when attacked. Organizations should therefore evaluate screens by their equilibrium performance against adapting authors, favor features that are costly to manipulate, and plan to maintain them. The evidence comes from three generators, one genre, and two judges, and a stronger adversary and measured cost parameters would be needed to turn these illustrations into estimates.

References

- Akerlof GA (1970) The market for “lemons”: Quality uncertainty and the market mechanism. *Quart. J. Econom.* 84(3):488–500.
- Cameron AC, Trivedi PK (2013) *Regression Analysis of Count Data*, 2nd ed. (Cambridge University Press, Cambridge, UK).
- Connelly BL, Certo ST, Ireland RD, Reutzel CR (2011) Signaling theory: A review and assessment. *J. Management* 37(1):39–67.
- Gehrmann S, Strobelt H, Rush AM (2019) GLTR: Statistical detection and visualization of generated text. *Proc. 57th Annual Meeting Assoc. Comput. Linguist.: System Demonstrations* (Association for Computational Linguistics, Florence, Italy), 111–116.
- Hevner AR, March ST, Park J, Ram S (2004) Design science in information systems research. *MIS Quart.* 28(1):75–105.
- Mitchell E, Lee Y, Khazatsky A, Manning CD, Finn C (2023) DetectGPT: Zero-shot machine-generated text detection using probability curvature. *Proc. 40th Internat. Conf. Machine Learn.* (PMLR).
- Pillutla K, Swayamdipta S, Zellers R, Thickstun J, Welleck S, Choi Y, Harchaoui Z (2021) MAUVE: Measuring the gap between neural text and human text using divergence frontiers. *Adv. Neural Inform. Processing Systems* 34.
- Spence M (1973) Job market signaling. *Quart. J. Econom.* 87(3):355–374.
- Wang RY, Strong DM (1996) Beyond accuracy: What data quality means to data consumers. *J. Management Inform. Systems* 12(4):5–33.

Zhang T, Kishore V, Wu F, Weinberger KQ, Artzi Y (2020) BERTScore: Evaluating text generation with BERT. *Proc. Internat. Conf. Learn. Representations*.

Zheng L, Chiang WL, Sheng Y, Zhuang S, Wu Z, Zhuang Y, Lin Z, Li Z, Li D, Xing EP, Zhang H, Gonzalez JE, Stoica I (2023) Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Adv. Neural Inform. Processing Systems* 36 (Datasets and Benchmarks Track).

Appendix A. Supplementary Tables

Table A1. AUC (95% bootstrap interval) for screens trained on human papers only. Scores are anomaly scores; test sets are held-out human papers (grouped by source paper) plus papers from the named generator. A Mahalanobis screen on the clustering features reaches 0.59, 0.49, and 0.65 on GPT-4o, Claude, and Llama papers.

Method (human-only)	Features	GPT-4o	Claude
Mahalanobis (primary)	Rates	0.597 [0.55, 0.64]	0.466 [0.42, 0.52]
Mahalanobis (primary)	Dispersion	0.507 [0.46, 0.55]	0.510 [0.45, 0.56]
Mahalanobis (primary)	Both	0.563 [0.52, 0.60]	0.462 [0.41, 0.52]
One-class SVM	Both	0.610 [0.56, 0.65]	0.475 [0.42, 0.53]
Isolation forest	Both	0.473 [0.43, 0.52]	0.468 [0.42, 0.52]

Table A2. Baseline versus scale-dependent clustering features, development data (exploratory). Baseline values differ from Table 3 by at most 0.001 because of a small change in how counts are split at window boundaries.

Train → Test	Baseline AUC	Clustering AUC	Difference [95% CI]
<i>Rates plus clustering features</i>			
GPT-4o → GPT-4o	0.987	0.991	+0.004 [−0.000, +0.008]
Claude → Claude	0.914	0.946	+0.032 [+0.017, +0.050]
GPT-4o → Claude	0.729	0.755	+0.026 [+0.001, +0.052]
Claude → GPT-4o	0.809	0.849	+0.040 [+0.019, +0.062]
<i>Clustering features only (no rates)</i>			
GPT-4o → GPT-4o	0.856	0.927	+0.071 [+0.045, +0.096]
Claude → Claude	0.751	0.794	+0.043 [+0.004, +0.082]
GPT-4o → Claude	0.704	0.730	+0.026 [−0.017, +0.071]
Claude → GPT-4o	0.808	0.858	+0.050 [+0.021, +0.077]

Table A3. Simulated cascade with bootstrap pools in which machine papers are 10% of the pool. Share of machine papers flagged when at most 5% of human papers are flagged, with 95% intervals over bootstrap pools. Sending 10% of documents detected 0.60, 0.30, and 0.66 for GPT-4o, Claude, and Llama papers. With about one third of the pool machine-written, detection at a 30% share falls to 0.76 for GPT-4o papers and 0.78 for Llama papers, and about 50% must be judged to recover 0.92 to 0.95.

Target generator	Share judged (q)	Judge cost per 1,000 docs	Detected, judge only	Detected, rank fusion
GPT-4o	20%	US\$3.5	0.86 [0.73, 1.00]	0.86 [0.73, 1.00]
	30%	US\$5.2	0.95 [0.86, 1.00]	0.96 [0.86, 1.00]
	50%	US\$8.7	0.96 [0.86, 1.00]	0.97 [0.91, 1.00]
	100%	US\$17.4	0.95 [0.82, 1.00]	0.97 [0.86, 1.00]
Claude	20%	US\$3.5	0.38 [0.11, 0.59]	0.38 [0.11, 0.59]
	30%	US\$5.2	0.36 [0.09, 0.59]	0.36 [0.09, 0.59]
	50%	US\$8.7	0.19 [0.05, 0.50]	0.35 [0.09, 0.59]
	100%	US\$17.4	0.12 [0.00, 0.32]	0.37 [0.09, 0.59]
Llama	20%	US\$3.5	0.82 [0.64, 0.95]	0.82 [0.64, 0.95]
	30%	US\$5.2	0.89 [0.73, 1.00]	0.89 [0.73, 1.00]
	50%	US\$8.7	0.97 [0.86, 1.00]	0.95 [0.82, 1.00]
	100%	US\$17.4	0.96 [0.86, 1.00]	0.93 [0.79, 1.00]